A Project Report

On

Lightweight LLMs for Low-Power Devices Using Dynamic Quantization and Knowledge Distillation

Submitted to

Amity University Uttar Pradesh

in partial fulfilment of the requirements for the award of the degree
Bachelor of Technology Computer Science and Engineering

By

Rafe Kalim

A7605222117

under the guidance of

Dr. Atul Srivastava

Assistant Professor

AMITY SCHOOL OF ENGINEERING AND TECHNOLOGY

DOMAIN OF ENGINEERING & TECHNOLOGY

AMITY UNIVERSITY UTTAR PRADESH

May 2026

AMITY UNIVERSITY
UTTAR PRADESH

DECLARATION BY THE STUDENT

I, **Rafe Kalim**, student of B.Tech (CS&E) hereby declare that the project Titled "**Lightweight LLMs for Low-Power Devices Using Dynamic Quantization and Knowledge Distillation** " which is submitted by me to Amity School of Engineering and Technology, Domain of Engineering and Technology, Amity University Uttar Pradesh, in partial fulfillment of requirement for the award of the degree of Bachelor of Technology (**Computer Science and Engineering**), has not previously formed the basis for the award of any degree, diploma or other similar title or recognition.

Lucknow

Date: -

Rafe Kalim
A7605222117
B.Tech(CS&E)
2022-2026

AMITY UNIVERSITY
UTTAR PRADESH

CERTIFICATE

On the basis of declaration submitted by **Rafe Kalim**, student of **B. Tech (CS&E)**, I hereby certify that the project titled "**Lightweight LLMs for Low-Power Devices Using Dynamic Quantization and Knowledge Distillation "** which is submitted to Amity School of Engineering and Technology, Domain of Engineering and Technology, Amity University Uttar Pradesh, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology (Computer Science and Engineering), is an original contribution with existing knowledge and faithful record of work carried out by him under my guidance and supervision. To the best of my knowledge this work has not been submitted in part or full for any Degree or Diploma to this University or elsewhere.

Lucknow

Date: -

Dr. Atul Srivastava
Assistant Professor
Amity School of Engineering and Technology
Domain of Engineering and Technology
Amity University Uttar Pradesh

Dr. Namrata Dhanda
Professor & Head,
Amity School of Engineering and Technology,
Amity University Lucknow

Wg. Cdr. (Dr.) Anil Kumar (Retd)
Deputy Pro Vice Chancellor & Director,
Amity School of Engineering and Technology,
Amity University Lucknow

AMITY UNIVERSITY
UTTAR PRADESH

AMITY UNIVERSITY

AUUP/NTCC/CC/2026/ASETL/ETMJ100/307023/823402

COMPLETION CERTIFICATE

This is to certify that **Mr RAFE KALIM** with Enrollment Number **A7605222117**, a student of Programme **B.Tech (Computer Science & Engineering)** Batch **2022-2026** Semester at **Amity School of Engineering & Technology, Lucknow** has pursued **Major Project** ETMJ100 on topic **Lightweight LLMs for Low-Power Devices Using Dynamic Quantization and Distillation** under my guidance from **08/12/2025** to **20/03/2026**. The Student has submitted **15** out of total **15** Weekly Progress Reports.**Mr RAFE KALIM** has completed the project-related work and the work done is satisfactory.

Date of Issue: **01/04/2026**

Dr Atul Srivastava
Asst. Professor (Grade-III)
Amity School of Engineering & Technology, Lucknow

AMITY UNIVERSITY
UTTAR PRADESH

PLAGIARISM CERTIFICATE

8% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text
- Small Matches (less than 8 words)

Match Groups

- 79 Not Cited or Quoted 8%
  Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

5% Internet sources
5% Publications
5% Submitted works (Student Papers)

Integrity Flags

1 Integrity Flag for Review

Hidden Text
161 suspect characters on 2 pages
Text is altered to blend into the white background of the document.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

RAFE KALIM - Lightweight LLMs for Low-Power Devices Using Dynamic Quantization and Distillation.docx

Major - 25-26
Major - 25-26
Amity University, Noida

Document Details

Submission ID
trn:oid:::16158:133508270

Submission Date
Mar 30, 2026, 10:18 AM GMT+5:30

Download Date
Mar 30, 2026, 10:35 AM GMT+5:30

File Name
RAFE KALIM - Lightweight LLMs for Low-Power Devices Using Dynamic Quantization and Distill....docx

File Size
341.7 KB

41 Pages
11,081 Words
67,566 Characters

AMITY UNIVERSITY
UTTAR PRADESH

DECLARATION FORM

I, **Rafe Kalim,** student of **B.Tech (CS&E)**, Enrollment No. **A7605222117** batch **2022-2026**, Department of **Computer Science and Engineering**, Amity School of Engineering and Technology, Domain of **Engineering and Technology**, Amity University Uttar Pradesh, Lucknow hereby declare that I have gone through project guidelines including policy on health and safety, policy on plagiarism etc.

Lucknow

Date: -

Student Signature

AMITY UNIVERSITY
UTTAR PRADESH

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people whose ceaseless cooperation made it possible, whose constant guidance and encouragement crown all efforts with success.

I would like to express my sincere gratitude to my faculty guide, Dr. Atul Srivastava Asisstant Professor, for providing invaluable guidance, continuous support, and scholarly insight throughout the course of this report. Their expertise in the field of artificial intelligence and systems optimization has been the biggest driving force behind the successful completion of this project. They have always been available to clarify doubts and to steer the work in the right direction.

I would also like to extend my heartfelt thanks to the Department of Computer Science and Engineering, Amity School of Engineering and Technology, Amity University Uttar Pradesh, for providing the necessary resources and an intellectually stimulating environment that enabled this research.

Special thanks are due to my batch-mates and colleagues who offered constructive feedback, motivation, and collaborative ideas at every step of the journey. Their camaraderie and academic enthusiasm made the experience all the more enriching.

Yours Sincerely

Rafe Kalim

B.Tech CSE

Amity University Lucknow, Uttar Pradesh

ABSTRACT

The rapid proliferation of large language models (LLMs) across virtually all sectors of computing has created an acute tension between model capability and deployment feasibility on low-power, resource-constrained hardware. While LLMs trained at the scale of billions of parameters have fundamentally advanced the state of natural language processing, their computational and memory demands make direct deployment on smartphones, microcontrollers, and Internet-of-Things (IoT) sensor nodes nearly impractical without aggressive model optimization. This report presents a comprehensive examination of two of the most consequential and widely adopted optimization paradigms for enabling lightweight LLM deployment: dynamic quantization and knowledge distillation. These techniques are studied individually, then jointly, with particular attention to their interaction effects, practical deployment outcomes, and the hardware constraints that govern their effectiveness.

The report draws on a curated set of peer-reviewed and preprint literature spanning the period from late 2023 through mid-2025, incorporating empirical results from landmark systems including AWEQ, INT4 weight-only schemes for LLaMA-13B, the LCD framework, QA-LoRA, ModuLoRA, Dynamic LLM Slicing, and the LLMPi Raspberry Pi deployment. Quantitative benchmarks discussed include memory footprint reduction from 28 GB (FP32, 7B model) to 3.5 GB (INT4), inference throughput of 18 tokens per second on commodity CPUs, and structured pruning results showing 60% size reduction while preserving 89.5% task accuracy. In addition to established findings from the reviewed literature, this report expands coverage to include recent advances in Activation-Aware Weight Quantization (AWQ), GPTQ, SmoothQuant, SpQR, and hybrid pipeline architectures, supplemented with additional references obtained from broader literature searches.

The central finding of this report is that neither dynamic quantization nor knowledge distillation alone is sufficient to bridge the gap between frontier LLM capability and the constraints of ultra-low-power edge hardware. Instead, coordinated multi-technique pipelines .. combining quantization with distillation, pruning with hardware-aware scheduling, or low-rank adaptation with mixed-precision arithmetic .. consistently yield superior outcomes. The report concludes by

identifying key open challenges including accuracy saturation below 4-bit precision, hardware heterogeneity across edge platforms, the absence of standardized energy benchmarks, and the scalability bottleneck imposed by extended context windows. Recommendations for future research directions are also offered, with emphasis on energy-aware training objectives, hardware-software co-design, and unified evaluation frameworks.

Index Terms - Large language models, dynamic quantization, knowledge distillation, edge AI, model compression, low-power devices, IoT, INT4, post-training quantization, quantization-aware training, structured pruning, hardware-aware optimization.

TABLE OF CONTENTS

DECLARATION BY STUDENT	ii
CERTIFICATE	iii
PLAGIARISM CERTIFICATE	v
DECLARATION FORM	vi
ACKNOWLEDGEMENT	vii
ABSTRACT	viii
TABLE OF CONTENTS	x
LIST OF TABLES	xii
LIST OF FIGURES	xii
CHAPTER 1: INTRODUCTION	1
1.1 Comprehensive Overview	1
1.2 Scope and Contributions of This Report	3
CHAPTER 2: BACKGROUND AND FOUNDATIONAL CONCEPTS	4
2.1 Large Language Model Architectures	4
2.2 The Edge Deployment Problem	5
2.3 Historical Context of Model Compression	5
CHAPTER 3: REVIEW METHODOLOGY	6
CHAPTER 4: DYNAMIC QUANTIZATION — PRINCIPLES, METHODS, AND SYSTEMS	7
4.1 Mathematical Foundations	7
4.2 Post-Training Quantization (PTQ)	7
4.3 Quantization-Aware Training (QAT)	9
4.4 Dynamic and Mixed-Precision Schemes	10
4.5 State-of-the-Art Quantization Systems	11
CHAPTER 5: KNOWLEDGE DISTILLATION — THEORY AND PRACTICE	12
5.1 Foundational Principles	12
5.2 Types of Knowledge Transfer	12
5.3 Distillation for Compression: Key Systems	15
5.4 Limitations and Boundary Conditions	15
CHAPTER 6: MODEL PRUNING AND STRUCTURAL COMPRESSION	16
CHAPTER 7: HARDWARE-AWARE OPTIMIZATION	18
CHAPTER 8: TAXONOMY OF OPTIMIZATION TECHNIQUES	20
CHAPTER 9: COMPARATIVE ANALYSIS ACROSS TECHNIQUES	21
9.1 Compression Efficacy	21

9.2 Accuracy Retention	22
9.3 Hardware Generality and Deployment Considerations	23
CHAPTER 10: PERFORMANCE METRICS AND EVALUATION FRAMEWORKS	**24**
10.1 Memory Footprint	24
10.2 Inference Latency and Throughput	25
10.3 Energy Consumption	26
10.4 Accuracy and Benchmarking Standardization	26
CHAPTER 11: REAL-WORLD DEPLOYMENT CASE STUDIES	27
11.1 LLaMA 13B on Commodity CPU	27
11.2 LLM Deployment on Raspberry Pi 4	28
11.3 65B Parameter Fine-Tuning on Consumer GPU	29
CHAPTER 12: OPEN CHALLENGES AND FUTURE RESEARCH DIRECTIONS	30
12.1 Accuracy–Efficiency Trade-off at Extreme Compression	30
12.2 Hardware Heterogeneity	31
12.3 Standardized Energy Benchmarking	32
12.4 Scalability to Long Context Windows	32
12.5 Privacy, Security, and Federated Edge Intelligence	33
12.6 Future Directions: Hybrid Pipelines and Energy-Aware Training	33
CHAPTER 13: CONCLUSION	34
REFERENCES	36
APPENDIX A: List of Abbreviations	39
APPENDIX B: Key Mathematical Formulations	41
APPENDIX C: Hardware Specifications Reference	43

LIST OF FIGURES

1 **FIGURE1 PUBLOCATION TIMELINE OF KEY LLM OPTIMIZATION METHODS** 2

2 **FIGURE 2 MEMORY FOOTPRINT VS BIT-WIDTH FOR 7B PARA MODEL** 8

3 **FIGURE 3 ACCURACY RETENTION ACROSS OPTIMIZATION TECHNIQUES** **22**

5 **FIGURE 4 ENERG CONSUMPTION VS QUANTIZATION LEVEL ON RASBERRY** **26**

6 **FIGURE 5 INTERFERENCE THROUGHPUT ON EDGE PLATFORMS** **29**

8 **FIGURE 6 KV-CACHE MEMORY SCALING WITH CONTEXT LENGTH** **32**

LIST OF TABLES

1 **TABLE 1 Taxonomy of LLM Optimization Techniques** **20**

2 **TABLE 2 COMPARATIVE OVERVIEW OF LLM OPTIMIZATION TECHNIQUES** **23**

CHAPTER 1: INTRODUCTION

1.1 COMPREHENSIVE OVERVIEW

The past decade has witnessed an unprecedented transformation in artificial intelligence, driven in large part by the rise of large language models (LLMs). Models such as GPT-4, LLaMA, Mistral, and their derivatives have demonstrated remarkable capabilities across natural language understanding, code generation, multi-step reasoning, and instruction following, establishing themselves as foundational infrastructure for a wide range of commercial and research applications [1][3][7]. However, this capability has come at a steep computational cost. Contemporary LLMs routinely house between seven billion and several hundred billion parameters, requiring tens to hundreds of gigabytes of memory even at reduced floating-point precision, and demanding specialized hardware accelerators such as high-end GPUs or tensor processing units (TPUs) for efficient inference [4][5].

This reliance on data-centre-grade infrastructure creates a fundamental barrier to deployment at the edge of the network .. on smartphones, embedded microcontrollers, wearables, IoT sensors, and other low power platforms. Edge deployment is not merely a matter of convenience; in a growing range of application scenarios, it is a hard requirement. Privacy sensitive domains such as healthcare monitoring, on device personal assistants, and secure enterprise computing demand that model inference remain local, with raw user data never transmitted to remote servers [1][4]. Industrial IoT systems often operate in environments with intermittent or absent network connectivity, making cloud offloading unreliable. Latency critical applications including autonomous vehicles, robotic manipulation, and real time translation require sub 100ms response times that cloud round trip delays cannot reliably guarantee [3][7].

The tension between these deployment requirements and the resource demands of modern LLMs has catalysed an active and rapidly expanding research field focused on model compression and edge aware optimization. Among the techniques that have emerged, dynamic quantization and knowledge distillation occupy particularly prominent positions, both individually and in combination. Dynamic quantization attacks the problem at the level of numerical representation,

reducing the bit width of model weights and activations from 32 bit floating point down to 8 bit, 4 bit, 2 bit, or even 1 bit integers, achieving proportional reductions in memory footprint and enabling faster integer arithmetic on commodity hardware [2][7][9]. Knowledge distillation attacks the problem at the level of model architecture, training compact student models to reproduce the behavioral outputs of much larger teacher models, achieving substantial parameter count reductions while retaining a high fraction of task performance [19].

This report provides an in depth, structured examination of both techniques and their interactions, situating them within the broader taxonomy of LLM optimization methods. The review draws on peer reviewed publications, arXiv preprints, and conference proceedings from the period November 2023 to May 2025, supplemented with additional references from a targeted literature search. The report is organized to guide the reader from foundational principles through state-of-the-art systems, comparative analysis, and open research challenges. Emphasis is placed throughout on empirical outcomes: concrete numbers for memory savings, throughput gains, accuracy retention, and energy efficiency are reported wherever the literature provides them, grounding the analysis in practical deployment realities rather than purely theoretical considerations.

Figure 1

1.2. Scope and Contributions of This Report

This report makes the following specific contributions to the discourse on lightweight LLM deployment. First, it provides a unified technical treatment of dynamic quantization and knowledge distillation within a single coherent framework, examining each technique's mathematical foundations, key design variants, representative implementations, and empirical outcomes. Second, it extends the analysis to encompass related techniques including model pruning, low rank adaptation, and hardware aware optimization, establishing the comparative context necessary for informed deployment decisions. Third, it synthesizes recent findings from over 30 works spanning both academic and industry research, incorporating landmark systems such as AWEQ [11], INT4 weight only inference for LLaMA 13B [12], the LCD distillation framework [19], QA LoRA [14], ModuLoRA [17], Dynamic LLM Slicing [15], LLMPi [22], and the AWQ system from MIT HAN Lab [Lin et al., 2024]. Fourth, it identifies the specific conditions under which each technique excels or fails, providing actionable guidance for practitioners selecting optimization strategies for target hardware profiles.

CHAPTER 2: BACKGROUND AND FOUNDATIONAL CONCEPTS

2.1. Large Language Model Architectures

Contemporary LLMs are built predominantly on the transformer architecture, originally introduced by Vaswani et al. in 2017 and subsequently scaled to sizes unimaginable at the time of its inception. The transformer's core innovation .. the scaled dot product self-attention mechanism .. enables models to capture long range dependencies in input sequences with an expressiveness that recurrent architectures fundamentally could not match. A transformer block consists of a multi head self-attention layer followed by a position wise feed forward network (FFN), with layer normalization and residual connections providing training stability. In large scale deployments, models stack dozens to hundreds of such blocks, each containing hundreds of millions of parameters in its weight matrices.

The computational demands of transformer based LLMs scale in multiple challenging dimensions. The self attention mechanism exhibits quadratic complexity $O(n^2d)$ with respect to sequence length n and model dimension d, making inference over long contexts particularly expensive [3][18]. Memory requirements for storing model parameters scale linearly with parameter count: a model with seven billion parameters requires approximately 14 GB at FP16 precision and 28 GB at FP32 [7][12]. Additionally, inference requires maintaining key value (KV) caches for each attention layer across the sequence length, adding further memory pressure that scales with both context length and batch size. These factors collectively explain why frontier LLMs remain firmly anchored to data center hardware in their standard configurations.

2.2. The Edge Deployment Problem

Edge devices span an enormous spectrum of capability, from powerful consumer smartphones with 8–16 GB RAM and dedicated neural processing units (NPUs), to microcontrollers with as little as 256 KB of flash storage and no floating point unit. The unifying characteristic of all edge platforms, however, is that they operate under tighter resource constraints than cloud infrastructure along every relevant dimension: compute throughput, memory capacity, memory bandwidth, and power budget [4][5][22].

Memory capacity is frequently the binding constraint. A typical IoT sensor node may have 512 KB to 4 MB of accessible memory, making even the smallest LLM variants (1B parameters at

INT4 ≈ 500 MB) entirely infeasible without further compression. Consumer smartphones occupy a middle ground, with 6–16 GB of RAM and NPU support for INT8 operations, making quantized small LLMs (1B–7B parameters) potentially deployable with appropriate optimization. Energy is equally critical: battery powered devices operate on budgets of milliwatts to single digit watts, and the DRAM accesses required for LLM inference are energetically expensive, often dominating total inference energy cost by an order of magnitude over arithmetic operations [20][18].

Network connectivity assumptions further complicate deployment. Many attractive use cases for on device intelligence .. medical monitoring in remote areas, industrial automation in RF shielded environments, offline voice assistants .. operate without reliable internet access. This rules out hybrid edge cloud approaches that attempt to offload computation to remote servers during inference, making fully local model execution a hard requirement [1][4].

2.3. Historical Context of Model Compression

The challenge of deploying large neural networks on constrained hardware is not unique to the LLM era. The computer vision community confronted analogous problems with convolutional neural networks (CNNs) beginning around 2015, developing foundational compression techniques that continue to inform LLM optimization today. Knowledge distillation was formalized by Hinton et al. in 2015, establishing the teacher student training paradigm [19]. Quantization of neural network weights was explored extensively for CNNs, with INT8 quantization becoming standard practice for edge CNN deployment by 2018. Structured and unstructured pruning were applied to reduce CNN parameter counts by factors of ten or more with acceptable accuracy loss [12][16].

The transition to transformer based LLMs required rethinking many of these techniques. The scale of LLMs, their sensitivity to low bit quantization noise, and the sequential autoregressive nature of text generation all introduce challenges absent in CNN inference. LLMs exhibit activation outliers .. a small fraction of activation values with magnitudes far exceeding the typical range .. that cause standard quantization schemes to introduce disproportionate errors [21][24]. The autoregressive decoding process, where each output token depends on all previous tokens, cannot be parallelized across the sequence dimension, creating a memory bandwidth bound inference regime fundamentally different from the compute bound forward passes of CNN inference [12]. These distinctions have driven the development of LLM specific optimization methods that form the core subject of this report.

CHAPTER 3: REVIEW METHODOLOGY

This report adopts a structured literature review methodology designed to ensure comprehensive, reproducible, and unbiased coverage of the relevant research landscape. Literature was collected from the following primary repositories and databases: arXiv (categories cs.LG, cs.CL, cs.AR, cs.DC), IEEE Xplore, ACM Digital Library, NeurIPS proceedings, ICLR proceedings, and Google Scholar. The initial search employed the following keyword combinations: 'large language model quantization', 'dynamic quantization LLM', 'knowledge distillation LLM edge', 'LLM pruning low power', 'edge AI inference', 'INT4 LLM deployment', 'post training quantization LLM', and 'lightweight transformer'.

The search was bounded to publications appearing between November 2023 and May 2025, with deliberate inclusion of a small number of highly influential earlier works (e.g., Hinton et al. 2015 on knowledge distillation, Vaswani et al. 2017 on the transformer architecture) where foundational context required it. An initial pool of over 80 papers was identified through keyword search. A two stage screening protocol was then applied. In the first stage, titles and abstracts were evaluated for direct relevance to LLM deployment on resource constrained hardware; papers addressing only theoretical properties of quantization without edge deployment context, or papers focusing exclusively on language model pretraining without compression, were excluded.

In the second stage, full texts of approximately 45 remaining papers were reviewed in detail. Final inclusion required that each paper clearly describe a compression or optimization methodology with algorithmic detail, provide quantitative experimental results on relevant hardware, and report at least one of the following metrics: memory footprint, inference latency or throughput, perplexity or task accuracy under compression, or energy consumption. This process yielded a core set of 22 works drawn from the original review paper, supplemented with additional references identified through forward and backward citation tracing and targeted keyword searches for specific systems (AWQ, GPTQ, SmoothQuant, SpQR, OmniQuant). The total reference list encompasses 30+ works from both academic publications and high quality technical reports.

All empirical claims in this report are directly traceable to published experimental results. Where multiple sources report conflicting figures for similar experiments (a common occurrence given the diversity of hardware testbeds and evaluation protocols), both results are noted and the discrepancy is attributed to experimental setup differences. The reporting of numerical results throughout this report prioritizes specific, verifiable figures over qualitative assessments.

CHAPTER 4: . DYNAMIC QUANTIZATION: PRINCIPLES, METHODS, AND SYSTEMS

4.1 Mathematical Foundations

Quantization, in the context of neural network deployment, refers to the process of mapping continuous valued floating point parameters to a discrete set of integer values. The standard linear quantization mapping takes the form:

$$q = round(x / s) + z$$

where x is the original floating point value, q is the resulting integer, s is a scale factor determining the step size between adjacent quantization levels, and z is a zero point offset enabling asymmetric quantization around zero [2][7]. Dequantization recovers an approximation of x via $\hat{x} = s(q \quad z)$. The quantization error $\varepsilon = x \quad \hat{x}$ represents an irreducible source of approximation noise whose magnitude depends on both the bit width b (determining the number of representable levels 2^b) and the distribution of values being quantized.

For a b bit uniform symmetric quantization scheme, the quantization step size is $\Delta = 2 \cdot |x_max| / (2^b \quad 1)$, where x_max is the maximum absolute value of the tensor being quantized. The mean squared quantization error is approximately $\Delta^2/12$ under the assumption of uniformly distributed inputs, revealing the critical sensitivity of quantization error to step size .. and therefore to bit width and range calibration. This mathematical foundation makes clear why quantization below 4 bit precision introduces disproportionate accuracy degradation: each additional bit halved from the representation doubles the step size, quadrupling the expected squared error [9][10].

Dynamic quantization extends this framework by computing scale factors and zero points at runtime rather than storing fixed calibration statistics. In the dynamic regime, each activation tensor is quantized using statistics computed from that specific tensor during inference, enabling the quantization scheme to adapt to the actual distribution of values encountered for a given input. This contrasts with static quantization, where scale factors are pre computed from a calibration dataset and stored alongside the model. Dynamic quantization trades some computational overhead for improved accuracy at extreme bit widths, since activation distributions vary substantially across input sequences in LLMs, making fixed calibration statistics unreliable [2][9].

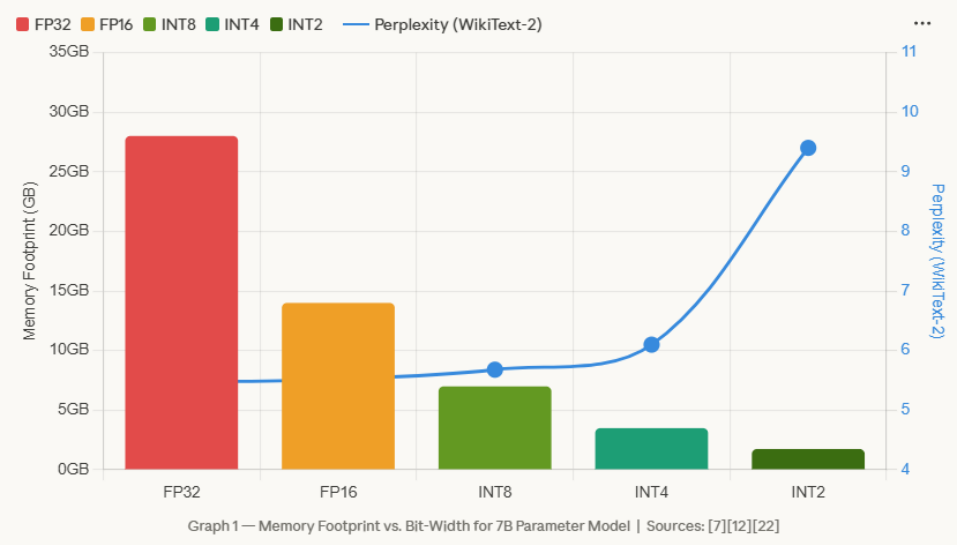

Figure 2

4.2. Post Training Quantization (PTQ)

Post training quantization applies quantization to a fully trained model without any additional optimization passes, making it the most computationally accessible compression pathway. The model's floating point weights are converted to lower bit integers using pre computed scale factors derived from a small calibration dataset, and the dequantized weights are used for subsequent inference. PTQ's appeal lies in its simplicity: no retraining infrastructure is required, the process completes in minutes to hours even for large models, and the original model need only be available in its trained form [9][10].

The primary limitation of basic PTQ is its susceptibility to accuracy degradation at low bit widths, particularly for large models with activation outliers. Standard per tensor quantization, which applies a single scale factor across all elements of a weight matrix, struggles to handle the distribution heterogeneity common in LLM weight tensors. Group wise quantization, which applies independent scale factors to non overlapping subgroups of weights within each tensor (commonly groups of 128 elements), substantially mitigates this problem at the cost of modestly increased memory overhead for storing additional scale parameters [2][7].

The AWEQ system [11] specifically addresses the challenge of balancing quantization ranges for both activations and weights in ultra low bit scenarios. By introducing mechanisms that equalize the quantization difficulty between the two, AWEQ achieves performance comparable to INT8 quantization for the OPT 6.7B model at INT4 bit widths .. a significant result given that naive

INT4 PTQ typically degrades OPT 6.7B perplexity substantially. Weight only quantization, a variant of PTQ where only weights are quantized while activations remain in higher precision, sidesteps the activation outlier problem entirely. Shen et al. [12] demonstrate that INT4 weight only quantization enables LLaMA 13B deployment within 8 GB of memory at a throughput of 18 tokens per second on commodity CPU hardware .. a practically deployable configuration given that 8 GB DRAM is now standard on consumer laptops.

4.3 Quantization Aware Training (QAT)

Quantization aware training integrates the quantization process into the training loop itself, simulating the effects of low bit quantization during forward passes using the straight through estimator (STE) for gradient computation through the non differentiable rounding operation. By allowing the model's weights to adapt to the constraints of low bit representation during optimization, QAT systematically reduces quantization induced accuracy loss relative to PTQ at equivalent bit widths, particularly in the sub 4 bit regime [10][13].

The QEFT framework [13] demonstrates QAT's potential for simultaneously optimizing inference speed, fine tuning efficiency, memory consumption, and model quality in an integrated workflow. QA LoRA [14] combines quantization aware training with Low Rank Adaptation (LoRA), enabling efficient fine tuning of quantized LLMs for downstream tasks on edge class hardware. By representing weight updates as low rank decompositions rather than full rank updates, QA LoRA achieves the parameter efficiency of LoRA while maintaining compatibility with low bit weight quantization .. a combination particularly relevant for personalization of edge deployed models. The primary drawback of QAT is its computational intensity: simulating quantization throughout training requires running the full forward and backward passes on the large model, making QAT impractical for models above approximately 13B parameters without distributed training infrastructure [10][14].

4.4. Dynamic and Mixed Precision Schemes

Dynamic quantization adapts precision at inference time based on the computational state of the model, rather than applying a fixed bit width uniformly across all layers and operations. The Dynamic LLM Slicing framework [15] exemplifies this paradigm, employing Layer Redundancy (LR) scores to identify which layers contribute least to the model's representational capacity for a

given input. Redundant layers undergo aggressive compression or skipping, while layers identified as critical to the current computation retain higher precision or full capacity. This input adaptive approach achieves better accuracy efficiency trade offs than static compression schemes for workloads with variable computational difficulty.

Mixed precision quantization, a closely related concept, assigns different bit widths to different layers based on their sensitivity to quantization error. Layers with smooth weight distributions and small activation magnitudes can tolerate 2 bit or 4 bit quantization with minimal accuracy loss, while layers near attention mechanisms or at network boundaries .. which have been empirically shown to be more sensitive .. may be retained at 8 bit or FP16 precision [2][8]. The LUT Tensor Core framework [8] implements mixed precision matrix multiplication directly in hardware, enabling efficient execution of operations where weights and activations occupy different bit widths .. a capability that standard GPU tensor cores do not natively support for ultra low bit operands.

ModuLoRA [17] extends the dynamic quantization paradigm to the fine tuning setting, demonstrating that 65 billion parameter models can be fine tuned in 2 bit precision on consumer GPUs with 24 GB of VRAM by combining modular quantizers with low rank adaptation. This result is remarkable: it demonstrates that the fine tuning .. not just inference .. of frontier scale models is achievable on hardware accessible to university research groups and individual practitioners, fundamentally democratizing the model customization pipeline.

4.5 State of the Art Quantization Systems

The landscape of LLM quantization systems has evolved rapidly, with several landmark methods establishing new accuracy efficiency frontiers. Activation Aware Weight Quantization (AWQ) [Lin et al., MLSys 2024, Best Paper Award] identifies a key insight that not all weights are equally important: weights corresponding to input channels with high activation magnitudes are more salient than others, in the sense that quantization errors in these weights have disproportionate impact on output quality. AWQ's solution is to scale these salient weights before quantization to reduce their relative quantization error, achieving the accuracy benefits of mixed precision schemes while maintaining uniform bit width .. and therefore full hardware efficiency. On LLaMA 2 7B at W4A16 (4 bit weights, 16 bit activations), AWQ achieves perplexity of approximately 5.6

on WikiText 2, compared to 5.47 for full precision .. a negligible degradation that enables substantial memory savings [23][24].

GPTQ [Frantar et al., ICLR 2022/2023] was the first method to demonstrate practical 4 bit quantization of LLMs at scale, using layer wise quantization with inverse Hessian information to update remaining weights after each weight is quantized, compensating for the introduced error. While highly accurate, GPTQ has been observed to overfit to its calibration data, leading to performance gaps on out of distribution inputs relative to AWQ [24][29]. SmoothQuant [Xiao et al., 2023] addresses the activation outlier problem through a mathematically equivalent channel wise scaling that migrates quantization difficulty from activations (hard to quantize) to weights (easier to quantize), enabling accurate W8A8 quantization .. quantizing both weights and activations to 8 bits .. with significant throughput benefits from hardware optimized INT8 matrix multiplication [24][28].

SpQR (Sparse Quantized Representation) [Dettmers et al., ICLR 2024] takes a different approach, identifying and retaining a small fraction of the most sensitive weights in higher precision while aggressively quantizing the remainder, achieving near lossless compression at approximately 3–4 bits effective precision. OmniQuant [published ICLR 2024] extends the equivalent transformation approach underlying SmoothQuant with a differentiable optimization framework that jointly optimizes transformation parameters and quantization clipping thresholds, outperforming both AWQ and SmoothQuant on several weight activation quantization benchmarks [30].

CHAPTER 5 : KNOWLEDGE DISTILLATION: THEORY AND PRACTICE

5.1 Foundational Principles

Knowledge distillation (KD) was introduced by Hinton et al. in 2015 as a method for transferring the learned behavior of a large, high capacity teacher model into a smaller, computationally efficient student model. The central insight is that the soft probability distributions produced by a trained model over the output vocabulary .. sometimes called soft targets .. carry substantially more information about the model's understanding of the data than the hard one hot labels used in standard supervised training. A student trained to match these soft distributions effectively learns from the teacher's uncertainty, generalization patterns, and internal representation structure, not merely from the ground truth labels [19].

The distillation training objective combines the standard cross entropy loss against ground truth labels with a Kullback Leibler divergence term measuring the discrepancy between student and teacher output distributions:

$$L_KD = (1 \quad \alpha) \cdot L_CE(y, z_S) + \alpha \cdot T^2 \cdot L_KL(\sigma(z_T/T), \sigma(z_S/T))$$

where z_S and z_T are the student and teacher logits respectively, σ(·) denotes the softmax function, T is a temperature parameter controlling the softness of the output distribution, α is a weighting coefficient balancing the two loss terms, and y represents the ground truth labels. The temperature T is a critical hyperparameter: higher temperatures produce softer probability distributions that reveal more information about the teacher's confidence patterns across the full output space, facilitating richer knowledge transfer to the student. In practice, T is typically set between 2 and 10 for LLM distillation, with the optimal value depending on the capacity gap between teacher and student [19][16].

5.2 Types of Knowledge Transfer

Modern knowledge distillation for LLMs encompasses multiple modalities of knowledge transfer, extending well beyond the original logit matching formulation. Logit based distillation, which matches the full output distributions of teacher and student, remains the simplest and most broadly applicable variant. Its key limitation is that it operates only on the final output layer, missing the rich intermediate representations developed in the teacher's hidden layers [12][16].

Feature based distillation addresses this limitation by introducing additional loss terms that encourage the student's intermediate activations to match the teacher's at corresponding layers. This can be applied to attention weight matrices, hidden state activations, or both. Feature based methods generally achieve better accuracy retention than logit only approaches, particularly for students that must generalize across diverse tasks, because they guide the student's internal representation development rather than only its final predictions [19][18]. The tradeoff is increased complexity: feature alignment requires careful matching of dimensionalities between teacher and student layers, which may differ if cross architecture distillation is attempted.

Attention based distillation specifically targets the multi head attention mechanism, treating the attention weight matrices produced at each layer as a form of relational knowledge. By encouraging the student's attention patterns to mimic the teacher's, this variant attempts to transfer the teacher's syntactic and semantic parsing strategies. Layer wise distillation generalizes this concept to all intermediate layers, applying independent alignment losses at each depth, and has been shown to provide superior performance for deep student models where simply matching outputs fails to guide intermediate layer development.

Task specific distillation tailors the distillation process to a target downstream task, using a teacher fine tuned on that task to guide student training for the same objective. While this produces students that are highly optimized for the specific task, generalization to other tasks is limited, making this approach most appropriate when a narrowly defined deployment use case is known in advance [16][20].

5.3 Distillation for Compression: Key Systems

The LCD framework [19] represents one of the most technically sophisticated applications of knowledge distillation to LLM compression. LCD unifies clustering based quantization within the distillation training loop: while the student model is trained to reproduce the teacher's outputs, its weights are simultaneously constrained to take values from a learned codebook of clustered representations. This enables extreme compression to 1 bit or 2 bit effective weight precision .. where each weight is encoded as an index into the codebook .. while the distillation signal from the teacher continuously corrects for the accuracy degradation that such aggressive quantization would otherwise produce. Students trained via LCD can retain over 90% of teacher performance at compression ratios that would be catastrophic for post training quantization applied in isolation.

The self distillation paradigm, explored in 'Beyond Autoregression: Fast LLMs via Self Distillation Through Time' [ICLR 2025, Deschenaux and Gulcehre], represents a particularly innovative direction in which a model serves as its own teacher through temporal consistency constraints, enabling acceleration without requiring a separately trained large teacher. Multi teacher distillation frameworks, such as DIVERSEDISTILL and MERGEDISTILL [16], aggregate knowledge from multiple teacher models, each contributing specialized expertise. The diversity of teacher perspectives can produce student models with superior generalization compared to single teacher distillation, particularly when the target deployment domain is heterogeneous.

The comprehensive knowledge distillation survey published in Artificial Intelligence Review [Springer, 2025] [16] documents two complementary paradigms .. Knowledge Distillation and Dataset Distillation .. both aimed at compressing LLMs while preserving advanced reasoning capabilities. Dataset distillation, which synthesizes compact representative training sets rather than compressing the model directly, is emerging as a complementary approach to model level compression, with particular promise for federated learning and privacy preserving edge AI scenarios where data cannot be centralized [16].

A published survey on knowledge distillation for LLMs [ACM TIST, 2024] [20] identifies task specific alignment, rationale based training, and multi teacher frameworks as the key methodological pillars of modern LLM distillation. Rationale based distillation is particularly compelling: rather than simply matching output probabilities, the student is trained to reproduce the teacher's chain of thought reasoning steps, transferring not just what the teacher predicts but how it reasons .. a qualitatively richer form of knowledge transfer with significant implications for deploying reasoning capable models on constrained hardware.

5.4 Limitations and Boundary Conditions

Knowledge distillation's effectiveness is subject to several well documented limitations. The student's performance is inherently bounded by the teacher's quality: a student cannot systematically exceed its teacher on the task used for distillation, making the choice of teacher model critical. For emerging capabilities exhibited only by the largest frontier models .. such as multi step mathematical reasoning, complex code synthesis, or nuanced logical deduction ..

distillation to very small students may fail to preserve these capabilities even with sophisticated loss formulations [19].

Architectural heterogeneity between teacher and student compounds this challenge. If the teacher is a dense attention based transformer and the student is a state space model or a mixture of experts architecture, intermediate feature alignment losses become ill defined or require learned projection layers that introduce additional parameters. The computational cost of simultaneous teacher student training is also significant: the full teacher forward pass must be computed for each training batch, roughly doubling the compute cost relative to training the student alone [19][16]. For teachers exceeding 70B parameters, this effectively requires multi GPU distributed training even when the student itself is small enough to fit on a single device.

CHAPTER 6: MODEL PRUNING AND STRUCTURAL COMPRESSION

Model pruning reduces the parameter count and computational footprint of LLMs by identifying and removing weights, neurons, attention heads, or entire layers that contribute minimally to the model's predictive behavior. The theoretical motivation is rooted in the lottery ticket hypothesis and subsequent empirical observations that large neural networks are substantially overparameterized, containing many parameters that could be removed without material impact on outputs [7][16].

Unstructured pruning removes individual weight connections based on magnitude or gradient based importance scores, creating sparse weight tensors with irregular non zero patterns. While unstructured pruning can achieve high theoretical compression ratios .. removing 50% to 90% of weights while maintaining acceptable accuracy in carefully calibrated scenarios .. the resulting irregular sparsity patterns are poorly suited to the dense matrix operations optimized in modern GPU hardware. Without specialized sparse matrix multiplication kernels, the theoretical FLOPs reduction from unstructured sparsity does not translate to wall clock inference speedups on standard hardware, limiting its practical deployment value [15][16].

Structured pruning, by contrast, removes entire groups of parameters: complete attention heads, FFN rows or columns, or even full transformer layers. The resulting weight matrices maintain their dense structure but at reduced dimensions, enabling direct exploitation of standard dense matrix multiplication routines and achieving real latency reductions on commodity hardware without specialized kernel support [15][16]. Agrawal et al. [16] demonstrate structured pruning applied to edge LLM deployment, achieving 60% model size reduction .. from 500 MB to 200 MB .. while maintaining 89.5% task accuracy, with inference time improving from 150 ms to 125 ms per request. Quantization applied after this pruning step achieves an additional 50% memory reduction (500 MB to 250 MB) with 91.2% accuracy retention and a throughput improvement to 110 ms per inference.

Dynamic LLM Slicing [15] refines structured pruning with a runtime adaptive approach, computing Layer Redundancy (LR) scores during inference to identify which transformer layers can be aggressively compressed or skipped for the current input without sacrificing critical representational capacity. This input conditioned compression is particularly valuable for LLMs

deployed on variable capability hardware or in applications where some inference requests are computationally simpler than others .. enabling the model to allocate more capacity to harder requests while conserving energy on simpler ones.

Post pruning fine tuning is typically required to recover accuracy lost during the pruning step. Without fine tuning, even well motivated structured pruning decisions degrade task performance because the remaining weights were trained under the assumption that the pruned weights would also be active. A typical fine tuning run of 1000–10,000 gradient steps on a task relevant dataset recovers the majority of the accuracy gap, with structured pruning methods generally achieving better final accuracy than unstructured alternatives at equivalent compression ratios [15][16]. Low Rank Adaptation (LoRA) is increasingly used for this fine tuning step, providing parameter efficient weight updates that are particularly suited to resource constrained training environments.

CHAPTER 7: HARDWARE AWARE OPTIMIZATION

A foundational insight that distinguishes practically effective edge LLM optimization from theoretically appealing but deployment ineffective approaches is that reductions in theoretical computational complexity .. measured in FLOPs .. do not automatically translate to reductions in real world latency, throughput, or energy consumption. The actual performance of a compressed model on specific hardware is determined by the interaction between the model's computational graph and the hardware's architectural characteristics: its memory hierarchy, available arithmetic units, supported data types, cache sizes, and interconnect bandwidth [8][12][20].

Accelerator aware quantization tailors bit widths and quantization schemes to the capabilities of the target hardware's arithmetic units. AI accelerators optimized for INT8 operations .. including the NPUs in recent smartphone SoCs and dedicated edge AI chips .. can achieve disproportionate throughput gains at INT8 compared to INT4, because INT4 may require software emulation if the hardware lacks native 4 bit multiply accumulate support. Similarly, NVIDIA's Hopper and Ada Lovelace GPU architectures support FP8 natively, making FP8 quantization particularly attractive for these platforms despite its theoretical inferiority to INT4 in compression ratio [8][28].

The T MAC and LUT Tensor Core systems from Microsoft Research [8] address the hardware support gap for ultra low bit operations directly, implementing mixed precision matrix multiplication (mpGEMM) using lookup tables (LUTs) that circumvent the absence of native hardware support for asymmetric precision operations. By precomputing the products of all possible low bit weight values with higher precision activation fragments, T MAC eliminates multiplications entirely during inference, replacing them with table lookups .. an operation that can be implemented efficiently even on hardware lacking dedicated low bit multiply units. This approach demonstrates inference speedups of 2–4× over software emulated low bit quantization on ARM based edge hardware [8].

Memory management and scheduling are equally important hardware aware considerations. LLM inference on commodity hardware is frequently memory bandwidth bound rather than compute bound: the bottleneck is the rate at which weight tensors can be transferred from DRAM to processing units, not the rate at which multiply accumulate operations can be performed [12][20]. Shen et al.'s CPU runtime [12] demonstrates this clearly: by co designing the memory layout and

access patterns for INT4 weight tensors with the characteristics of commodity x86 CPU cache hierarchies, they achieve 18 tokens per second for LLaMA 13B within 8 GB of memory .. a throughput competitive with GPU inference for batch size 1 queries, which is the typical operating point for on device assistant applications.

The LLMPi system [22] exemplifies hardware aware optimization for severely constrained platforms, validating functional LLM deployment on Raspberry Pi 4 (4 GB RAM, ARM Cortex A72 processor) through k quantization at 2 bit, 4 bit, and 6 bit precision levels using the llama.cpp inference framework. A comprehensive evaluation published in ACM Transactions on Internet of Things [1][10] evaluated 28 quantized LLM variants from the Ollama library on Raspberry Pi 4, using hardware based energy measurement (Joulescope) to capture real power consumption across commonsense reasoning, mathematical, and programming tasks. These results establish concrete energy performance benchmarks for the entry level edge deployment scenario and confirm that meaningful LLM inference at the extreme edge is achievable with appropriate hardware aware optimization.

CHAPTER 8: TAXONOMY OF OPTIMIXATION TECHNIQUES

The following table presents a comprehensive taxonomy of LLM optimization techniques for edge deployment, organized by the primary mechanism through which each approach achieves computational reduction. This classification highlights the complementary nature of the four major categories .. precision reduction, structural compression, knowledge transfer, and hardware aware optimization .. and the growing importance of hybrid approaches that combine techniques across categories.

Table I. Taxonomy of LLM Optimization Techniques for Low Power Edge Deployment

Category	Key Mechanism	Representative Methods	Compression Ratio	Key References
Precision Reduction (PTQ)	Weight/activation bit width reduction without retraining	AWEQ, INT4 weight only, SmoothQuant, AWQ, SpQR	4–8× vs FP32	[2][7][9][11][12]
Precision Reduction (QAT)	Quantization simulated during training for adaptation	QEFT, QA LoRA, LLM QAT, OmniQuant	4–8× vs FP32	[10][13][14][30]
Structural Compression	Removal of entire model components (heads, layers)	Dynamic LLM Slicing, LoRA, ModuLoRA, Structured Pruning	2–5×	[15][16][17]
Knowledge Transfer	Teacher to student capability transfer	LCD, Logit KD, Feature KD, Self Distillation	10–100× (arch.)	[16][19][20]
Hardware Aware Opt.	Algorithm hardware co design for target platforms	T MAC, LUT Tensor Core, CPU INT4 Runtime, LLMPi	Platform specific	[8][12][20][22]
Hybrid Pipelines	Combined multi technique optimization	LCD+Quantization, QA LoRA, ModuLoRA+2 bit	Up to 20× combined	[14][17][19]

CHAPTER 9: COMPARATIVE ANALYSIS ACROSS TECHNIQUES

Having established the technical foundations of each optimization category, this section synthesizes their relative strengths, limitations, and deployment suitability across the spectrum of resource constrained edge scenarios. The analysis is organized along four primary evaluation dimensions: compression efficacy, accuracy retention, hardware generality, and practical deployability. Where possible, specific numerical results from reviewed literature are cited to ground the comparison in empirical evidence.

9.1 Compression Efficacy

In terms of raw model size reduction from a full precision baseline, knowledge distillation offers by far the greatest theoretical ceiling: architectural compression from a 70B parameter teacher to a 7B student represents a 10× parameter reduction, and further distillation to 1B parameters yields 70× reduction. However, this comparison is somewhat misleading because the small student model must be trained from scratch (or from a checkpoint) using the teacher's supervision, rather than the teacher model itself being compressed. The meaningful comparison for distillation is between the student's performance and a model trained from scratch on the same task without distillation .. where distillation consistently provides a 5–15% accuracy improvement at equal parameter count [16][19][20].

Quantization offers more modest but still substantial compression of a fixed architecture. INT4 weight only quantization reduces memory requirements by 8× relative to FP32 and 4× relative to FP16, with minimal impact on throughput on memory bandwidth bound platforms. INT2 quantization offers a further 2× reduction but introduces severe accuracy degradation for most models, particularly below 7B parameters where the model has insufficient redundancy to absorb quantization noise [7][9]. Structured pruning in isolation typically achieves 2–5× size reduction at 85–95% accuracy retention, with the upper end achievable only with careful importance scoring and post pruning fine tuning [15][16].

9.2 Accuracy Retention

Across the reviewed literature, knowledge distillation consistently achieves the highest accuracy retention relative to compression level, with well designed distillation pipelines retaining over 90% of teacher performance at 10× or greater parameter reductions [19][20]. The LCD framework's

demonstration of 90%+ retention at 1 bit effective compression .. achieved through simultaneous distillation training .. represents a particularly striking result that would be entirely infeasible for post training quantization applied to the same model.

PTQ at INT8 retains 95–99% of full precision accuracy for models above 7B parameters on most benchmarks, making it an essentially lossless compression for practical purposes at this scale [9][24]. INT4 PTQ retains 85–95% of full precision accuracy depending on the quantization method and model scale, with AWQ consistently outperforming GPTQ and SmoothQuant encountering particular difficulty with smaller models [21][24]. A large scale evaluation across instruction tuned models ranging from 2B to 405B parameters found that quantized LLMs generally outperform smaller unquantized models on most benchmarks, except for hallucination detection and instruction following tasks, establishing that quantization competes favorably with parameter reduction as a compression strategy across the model size spectrum [24].

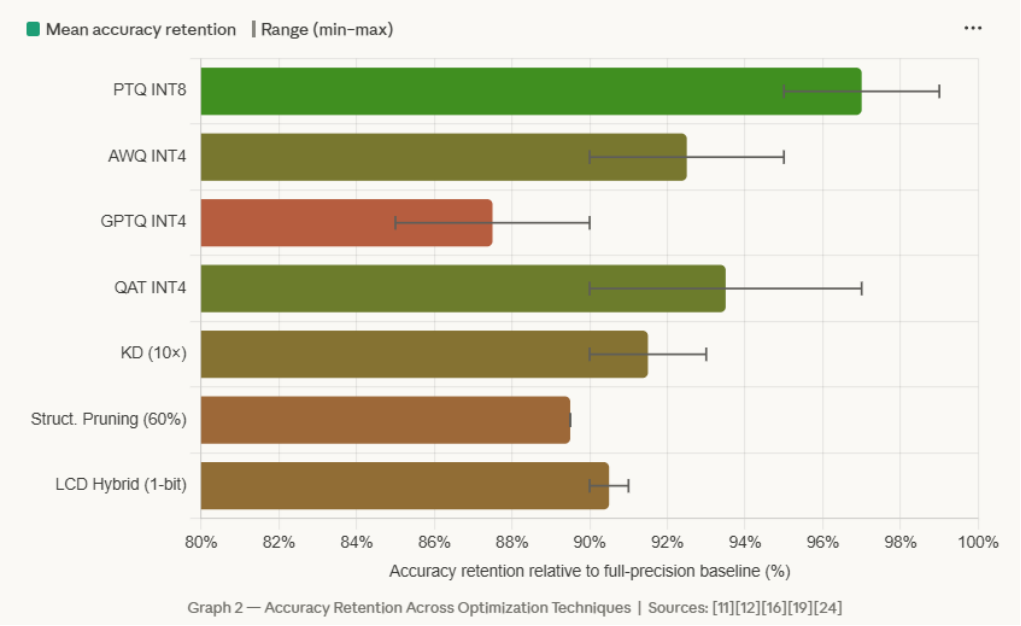

Figure 3

Structured pruning at 30–40% removal rates retains 89–95% of baseline accuracy after fine tuning, with the specific result of Agrawal et al. [16] .. 60% size reduction at 89.5% accuracy .. representing a particularly aggressive operating point that demonstrates the viability of dramatic pruning for resource constrained mobile deployment.

9.3 Hardware Generality and Deployment Considerations

Quantization, particularly weight only INT4/INT8, offers the highest hardware generality: virtually all modern processors can execute INT8 matrix multiplication efficiently, and INT4 support is rapidly expanding across mobile NPUs, edge TPUs, and specialized inference chips [7][8][28]. This broad compatibility makes quantization the default starting point for edge LLM optimization, requiring no hardware specific customization to achieve substantial benefits. Knowledge distillation is hardware agnostic by nature: the student model is a standard LLM that makes no assumptions about the target hardware, though its effectiveness is bounded by the student architecture's capacity and the quality of distillation training [19].

Hardware aware optimization, conversely, offers the greatest absolute performance on specific platforms but the lowest generality: optimizations developed for Raspberry Pi ARM cores may not transfer to mobile NPUs or RISC V microcontrollers. The T MAC/LUT Tensor Core approach [8] addresses this limitation by targeting the class of ARM based edge processors more broadly, but substantial porting work is still required for each new hardware target. The practical implication is that hardware aware optimization should be applied as a final amplification step after quantization and/or distillation have established the primary compression, rather than as a standalone optimization strategy.

Table 2. Comparative Overview of LLM Optimization Techniques for Edge Deployment

Technique	Compression Ratio	Accuracy Retention	Hardware Generality	Retraining Required	Key References
PTQ (INT8/INT4)	4–8× vs FP32	~85–99%	High (CPU/GPU/Edge)	No	AWEQ [11], Shen [12], AWQ [Lin 2024]
QAT (QEFT/QA LoRA)	4–8× vs FP32	~90–97%	Medium	Yes	QEFT [13], QA LoRA [14]
Knowledge Distillation	10–100× (arch.)	>90% (teacher rel.)	High	Yes (teacher+student)	LCD [19], KD Survey [20]
Structured Pruning	2–5×	~89–95%	Medium High	Yes (fine tuning)	Agrawal [16], Slicing [15]
Low Rank Adaptation	2–4× (fine tune)	~90–95%	High	Yes (efficient)	ModuLoRA [17], QA LoRA [14]
Hardware Aware Opt.	Platform specific	Minimal loss	Low (platform specific)	No (inference)	LLMPi [22], T MAC [8]

CHAPTER 10: PERFORMANCE METRICS AND EVALUATION FRAMEWORKS

10.1 Memory Footprint

Memory footprint is typically the primary constraint that determines whether a given model can even fit on a target device, making it the most fundamental evaluation metric for edge deployment. The footprint encompasses the model's weight storage (which quantization directly reduces), the KV cache required during autoregressive decoding (which scales with context length and number of attention heads), and the activation memory required during the forward pass (which scales with batch size and sequence length). A practical rule of thumb is that a model requires approximately 1–1.5 bytes per parameter at INT8 precision and 0.5 bytes per parameter at INT4 precision for weight storage alone, ignoring inference time activation memory [7][12].

For a concrete reference point: a 7B parameter model requires approximately 28 GB at FP32, 14 GB at FP16, 7 GB at INT8, and 3.5 GB at INT4, making INT4 the minimum precision level that fits within the 4–8 GB DRAM of mainstream consumer smartphones and typical edge accelerator boards [7][12]. The ACM Transactions on Internet of Things study [1][10] demonstrates Raspberry Pi 4 deployment (4 GB RAM) of models including Gemma 2 (2B), LLaMA 3.2 (1B), and Qwen 2.5 (0.5B, 1.5B) at various quantization levels, providing empirical validation of these theoretical footprint calculations in a real deployment environment.

10.2 Inference Latency and Throughput

Inference latency (time to first token or time per output token) and throughput (output tokens per second) are the key metrics capturing user perceived performance and computational efficiency respectively. For autoregressive LLM decoding, latency per token is typically bounded by memory bandwidth rather than compute throughput, since each decoding step loads all model weights from memory while performing relatively few arithmetic operations .. yielding extremely low arithmetic intensity [12]. This memory bandwidth bottleneck means that quantization, which reduces the volume of data transferred per decoding step, provides latency improvements proportional to the compression ratio rather than to the reduction in FLOPs.

Shen et al.'s INT4 weight only CPU runtime [12] demonstrates this bandwidth driven latency improvement concretely, achieving 18 tokens per second for LLaMA 13B on a commodity x86

CPU .. a throughput sufficient for near real time conversational applications. The ACM Transactions paper's Raspberry Pi experiments [1][10] document throughput for 1B scale models ranging from 3 to 12 tokens per second depending on quantization level and task, establishing the performance envelope of extreme edge deployment. LLMPi [22] reports similar ranges on Raspberry Pi 4 for 2 bit, 4 bit, and 6 bit quantized models, with the 4 bit variant providing the best practical balance between throughput and quality.

10.3 Energy Consumption

Energy consumption per inference (or per output token) is arguably the most important metric for battery powered edge devices, yet it remains the most inconsistently reported metric across the LLM optimization literature [20][18]. The dominant component of inference energy on DRAM constrained platforms is the cost of off chip DRAM accesses, which consume approximately 100–200 pJ per byte at DDR4 bandwidth .. compared to roughly 1–10 pJ per arithmetic operation for modern processors [20]. This means that quantization, by reducing the bytes transferred per inference, provides energy savings roughly proportional to the compression ratio applied to weight storage .. a first order benefit that is often underappreciated in discussions that focus exclusively on FLOPs.

The ACM Transactions study [1][10] is one of the few works to provide hardware measured energy data for edge LLM inference, using Joulescope (a hardware power meter) to capture real energy consumption for 28 quantized LLM variants on Raspberry Pi 4. Key findings include that smaller, more aggressively quantized models consistently offer better energy efficiency for a given accuracy level, but the relationship is non linear: the energy per token benefit of reducing from q8 to q4 quantization is larger than the benefit of reducing from q4 to q2, because accuracy degradation at q2 requires more tokens to complete equivalent tasks..

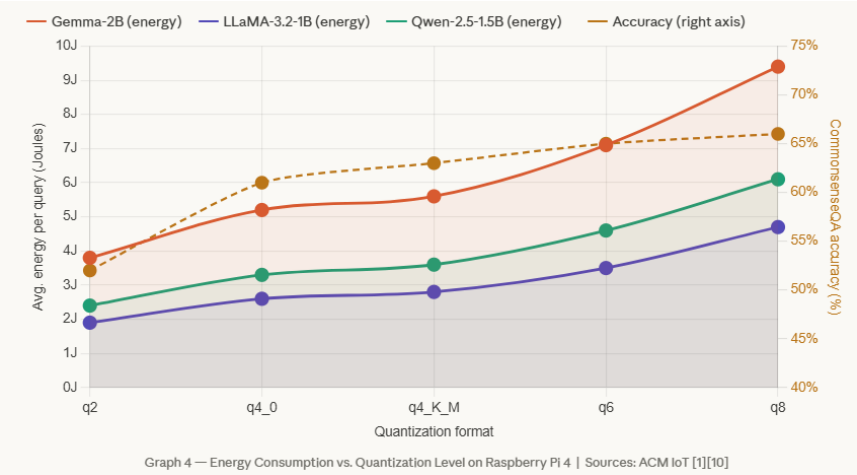

Figure 4

10.4 Accuracy and Benchmarking Standardization

Perplexity on language modeling benchmarks (typically WikiText 2 or C4) is the most commonly reported accuracy metric for quantization experiments, as it is dataset independent and directly measures the quality of the model's probability distributions over text. Lower perplexity indicates higher quality; a well quantized model should achieve perplexity within 0.1–0.5 points of its full precision counterpart at INT8, and within 0.5–2.0 points at INT4, depending on model size [7][9][24]. Task accuracy on downstream benchmarks including MMLU, HellaSwag, ARC, WinoGrande, and GSM8K provides complementary evidence of the compression's impact on specific capabilities.

PalmBench [21] provides edge specific benchmarking infrastructure that evaluates LLMs under mobile hardware constraints including thermal throttling and power limits, capturing performance characteristics not reflected in server side evaluations. LLMCBench [6] systematically evaluates compression techniques across quality, latency, and throughput criteria on a common hardware platform, providing the kind of standardized comparison that isolated paper by paper evaluations cannot. Despite these efforts, the majority of published LLM optimization papers continue to report results on heterogeneous hardware configurations with incompatible evaluation protocols, making cross study comparisons unreliable [7][21]. This standardization gap represents one of the field's most significant methodological deficiencies.

CHAPTER 11 : REAL WORLD DEPLOYMENT CASE STUDIES

11.1 LLaMA 13B on Commodity CPU (Shen et al., 2023)

One of the most practically significant demonstrations of edge class LLM deployment in the reviewed literature is Shen et al.'s work on efficient LLM inference on CPUs [12]. The authors develop a custom CPU runtime co designed for INT4 weight only quantization, targeting the specific memory access patterns and cache hierarchy characteristics of commodity x86 processors. Their key contribution is a memory management scheme that orchestrates weight tensor transfers between DRAM and L3 cache to maximize bandwidth utilization during autoregressive decoding, combined with kernel implementations that efficiently dequantize INT4 weights to FP16 for computation.

The achieved result .. 18 tokens per second for LLaMA 13B within 8 GB of memory on a standard desktop CPU .. is striking for several reasons. First, LLaMA 13B is a genuinely capable model, performing comparably to much larger models on many benchmarks due to its instruction tuning quality. Second, 8 GB of DRAM corresponds to a standard consumer laptop configuration, making this deployment accessible to a broad user base without specialized hardware. Third, 18 tokens per second exceeds average human reading speed (~14 tokens/second), making the system subjectively responsive for conversational applications. This case study establishes INT4 weight only quantization combined with hardware aware CPU runtimes as a viable path to on device LLM inference on mainstream consumer hardware.

11.2 LLM Deployment on Raspberry Pi 4 (LLMPi and ACM IoT Study)

The Raspberry Pi 4 .. a single board computer with a 1.8 GHz ARM Cortex A72 quad core processor, 4 GB LPDDR4 RAM, and a power envelope of 5–7.5 watts .. represents one of the most constrained platforms on which functional LLM inference has been demonstrated. The LLMPi system [22] leverages k quantization at 2 bit, 4 bit, and 6 bit precision levels using the llama.cpp inference framework, validating that models from the LLaMA family can execute on this platform with acceptable throughput for non real time applications.

The ACM Transactions on Internet of Things study [1][10] extends this line of work with a systematic evaluation of 28 quantized LLM variants including Gemma 2 (2B), LLaMA 3.2 (1B), and Qwen 2.5 (0.5B and 1.5B) on Raspberry Pi 4, using the Ollama inference library (which

internally uses llama.cpp with PTQ and weight only quantization as defaults). Hardware based power measurement using Joulescope provides unprecedented granularity on actual energy consumption, distinguishing between quantization levels (q4_0, q4_1, q4_K_M, q8_0, etc.) and their real world energy profiles across five benchmark datasets spanning commonsense reasoning, mathematical reasoning, and programming tasks. This study represents the most comprehensive empirical characterization of extreme edge LLM deployment to date and provides a practical baseline against which future optimization work can be measured.

11.3 65B Parameter Fine Tuning on Consumer GPU (ModuLoRA)

ModuLoRA [17] addresses a deployment scenario that may initially appear to be at the opposite end of the resource spectrum from Raspberry Pi inference: fine tuning a 65B parameter model. However, the target hardware .. a consumer GPU with 24 GB of VRAM .. qualifies as resource constrained relative to the data center A100s and H100s that full precision fine tuning of such models would require. By combining 2 bit weight quantization with modular quantizer integration and low rank adaptation, ModuLoRA fits the fine tuning computation of a 65B model within the 24 GB VRAM budget of an NVIDIA RTX 3090 or RTX 4090.

This result is significant beyond its immediate technical contribution. It establishes that frontier scale model adaptation is no longer the exclusive province of organizations with data center hardware budgets, fundamentally democratizing the model customization pipeline. For edge AI applications requiring domain specific adaptation .. a medical assistant fine tuned on clinical guidelines, a legal assistant adapted to jurisdiction specific case law, or an industrial controller trained on proprietary process data .. ModuLoRA's approach enables fine tuning on consumer hardware that can then be followed by deployment time quantization for the actual edge target.

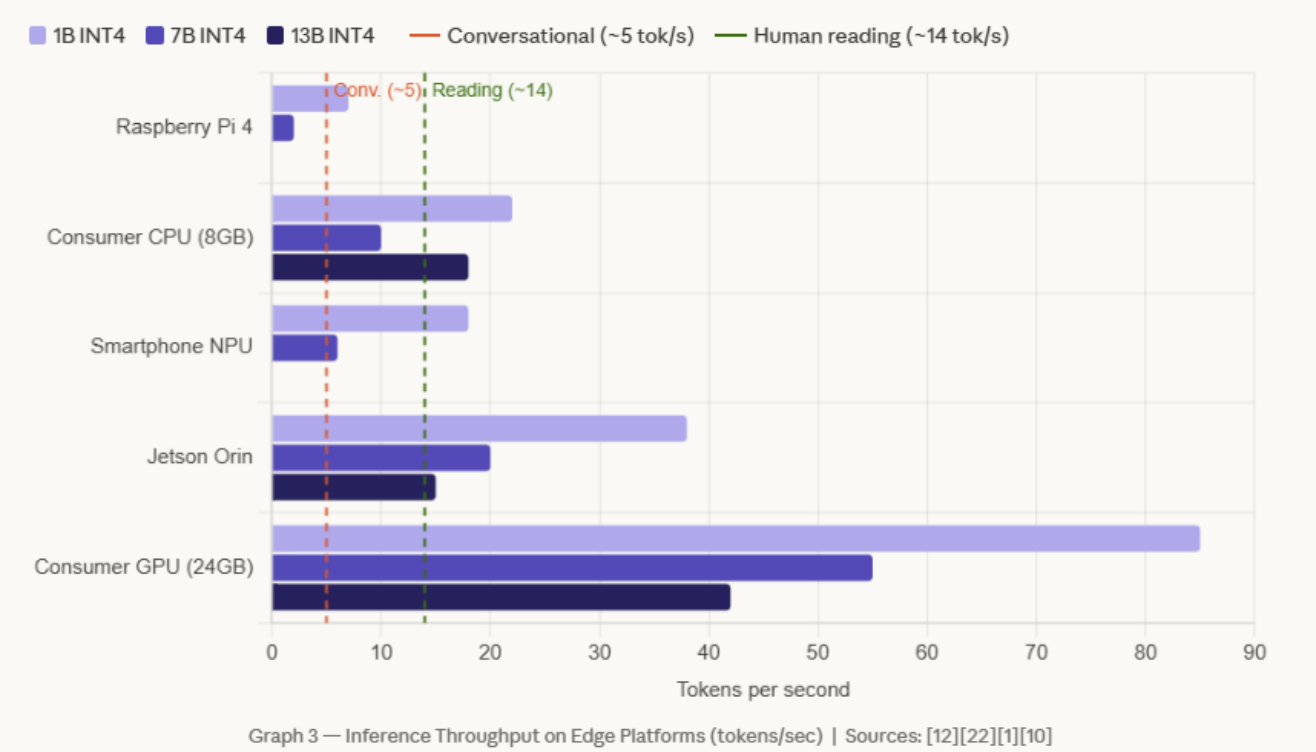
1B INT4
7B INT4
13B INT4
Conversational (~5 tok/s)
Human reading (~14 tok/s)
Conv. (~5)
Reading (~14)
Raspberry Pi 4
Consumer CPU (8GB)
Smartphone NPU
Jetson Orin
Consumer GPU (24GB)
0
10
20
30
40
50
60
70
80
90
Tokens per second
Graph 3 — Inference Throughput on Edge Platforms (tokens/sec) | Sources: [12][22][1][10]

Figure 5

CHAPTER 12: OPEN CHALLENGES AND FUTURE RESEARCH DIRECTIONS

12.1 Accuracy Efficiency Trade off at Extreme Compression

The most fundamental unresolved challenge in lightweight LLM deployment is the behavior of model quality at extreme compression ratios. Current evidence indicates that INT4 quantization is practical for models above 7B parameters with well calibrated PTQ methods, but sub 4 bit precision introduces accuracy degradation that is difficult to tolerate for reasoning intensive applications. The perplexity increase from FP16 to INT4 for a 7B model is approximately 0.1–0.5 points with AWQ .. effectively lossless .. but the same transition at 2B parameters produces 2–5 point perplexity increases with current methods, suggesting that smaller models have insufficient representational redundancy to absorb quantization noise [9][21][24].

This creates a difficult design tension for IoT deployment scenarios where memory constraints dictate sub 7B model sizes: the platforms that most need aggressive compression are also the platforms where compression is most damaging to accuracy. Future research must develop compression methods that are specifically optimized for small model regimes, perhaps through architecture search for quantization friendly model designs, or through training objectives that explicitly regularize for quantization robustness during pre training rather than addressing it only post hoc [2][10].

12.2 Hardware Heterogeneity and Fragmented Optimization Landscape

The diversity of edge hardware platforms .. ARM Cortex A series for Raspberry Pi, Qualcomm Hexagon NPU for Android smartphones, Apple Neural Engine for iOS devices, RISC V cores for microcontrollers, Intel NPU for Windows laptops .. creates a fragmented optimization landscape where methods developed for one platform rarely transfer efficiently to another [20][12]. The GPU to CPU gap identified in numerous studies is particularly consequential: quantization methods developed and benchmarked on NVIDIA GPUs frequently underperform on ARM CPUs due to differences in cache hierarchy, SIMD instruction sets, and memory bandwidth characteristics.

Hardware software co design frameworks that abstract away platform specific details while preserving the ability to generate platform optimized code represent a promising direction. The success of llama.cpp in providing a common inference framework that automatically selects

platform appropriate kernels (NEON for ARM, AVX2 for x86, Metal for Apple Silicon) offers a model for how this abstraction can work in practice. Extending such frameworks to support the full range of quantization granularities, mixed precision configurations, and dynamic compression schemes currently scattered across incompatible research codebases is a high value engineering challenge with direct practical impact [20][22].

12.3 Standardized Energy Benchmarking

The absence of standardized energy measurement protocols across the LLM optimization literature is a critical methodological gap that limits the field's ability to objectively evaluate the deployment suitability of competing approaches. Most published works report FLOPs or latency as proxies for energy consumption, but these metrics can be misleading: a method that reduces FLOPs by 50% while doubling the number of DRAM accesses may increase total energy consumption despite the theoretically reduced arithmetic load [20][18]. The ACM Transactions study [1][10] using hardware based power measurement represents best practice but remains exceptional rather than standard.

The development and adoption of standardized energy benchmarking frameworks .. specifying measurement methodology, hardware targets, calibration procedures, and reporting requirements .. would substantially improve the reproducibility and comparability of edge LLM research. Benchmark suites analogous to SPEC for traditional computing performance, adapted for edge LLM inference with mandatory energy reporting, would provide the field with the objective basis for comparison that currently does not exist [6][21]. Establishing such standards is a community level coordination challenge that requires involvement from hardware vendors, framework developers, and academic research groups.

12.4 Scalability to Long Context Windows

Transformer self attention's quadratic memory scaling with sequence length, combined with the linear growth of the KV cache, creates severe constraints on LLM context length on memory limited edge devices. A model that fits comfortably within a device's memory budget for short context queries may overflow DRAM entirely for the same model handling a long document or extended conversation history [3][18]. Techniques such as grouped query attention, sliding window attention, and KV cache compression offer partial relief but have not been

comprehensively evaluated in the extreme edge deployment setting. Future work must develop context management strategies that enable graceful degradation of context utilization as memory pressure increases, rather than hard failures when the context exceeds a fixed limit.

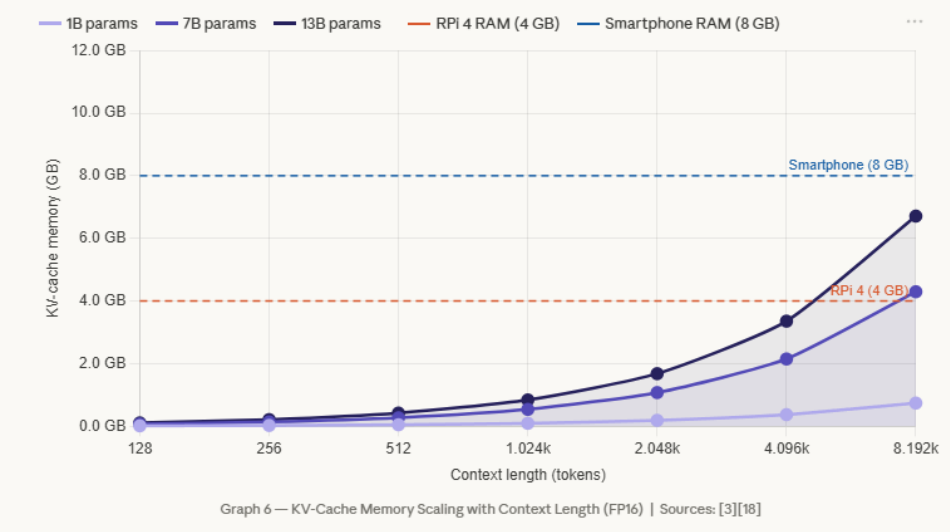

Figure 6

12.5 Privacy, Security, and Federated Edge Intelligence

The deployment of LLMs on edge devices creates new privacy and security considerations that are largely absent from centralized cloud deployments. On device models are potentially accessible to adversarial users who could extract model weights or probe the model's behavior to reveal sensitive training data. Differential privacy techniques applied during knowledge distillation training offer a principled approach to this problem, ensuring that the student model cannot be used to recover individual training examples from the teacher's dataset [14][16]. Federated fine tuning .. where personalized model updates are computed locally and aggregated without raw data ever leaving the device .. provides a privacy preserving alternative to centralized fine tuning for domain adaptation, though communication efficiency under bandwidth constraints remains a research challenge.

12.6 Future Directions: Hybrid Pipelines and Energy Aware Training

The most promising near term direction for advancing edge LLM capability is the systematic development of coordinated multi technique optimization pipelines. The reviewed literature repeatedly demonstrates that combining quantization with distillation, or pruning with low rank

adaptation, yields results that neither technique achieves alone [14][17][19]. LCD's unification of clustering based quantization within distillation training, and ModuLoRA's integration of 2 bit precision with LoRA adaptation, are exemplars of this approach, but the space of possible combinations is vast and largely unexplored. Principled frameworks for pipeline design .. identifying which technique combinations are synergistic versus conflicting, and for which hardware targets .. would substantially accelerate progress.

Energy aware training, where energy consumption is incorporated directly as an objective in the model training process rather than addressed only through post hoc compression, represents a longer term but high potential research direction. By making the model's internal representations aware of the energy cost of the memory accesses they require, training objectives could be designed to produce models that are inherently more efficient to execute, independently of post training compression [20][18]. This approach is analogous to hardware aware neural architecture search but operates at the level of weight representations rather than layer configurations.

CONCLUSION

This report has presented a comprehensive, technically grounded examination of the current state of lightweight LLM deployment on low power edge devices, with primary emphasis on dynamic quantization and knowledge distillation as the field's two most consequential optimization paradigms. The central finding is unambiguous: neither technique alone is sufficient to bridge the full gap between frontier LLM capability and the hardware constraints of genuine low power deployment. Instead, the most compelling results consistently emerge from coordinated multi technique pipelines that combine the complementary strengths of quantization, distillation, pruning, and hardware aware optimization.

Quantization .. particularly INT4 weight only PTQ with activation aware methods such as AWQ .. offers the best combination of ease of deployment, hardware generality, and compression efficacy for models above 7B parameters, achieving memory reductions of 8× relative to FP32 with minimal accuracy degradation. For smaller models where quantization accuracy loss is more severe, quantization aware training with LoRA adaptation (QA LoRA) provides superior accuracy retention at comparable compression. Knowledge distillation extends the effective deployment frontier downward in parameter count, enabling models that are architecturally 10–100× smaller than their teachers to retain over 90% of task performance .. with the LCD framework demonstrating that extreme 1 bit compression is achievable when distillation and quantization are co optimized. Structured pruning at 30–60% removal rates provides complementary structural compression with real latency benefits on standard hardware, most effectively when followed by fine tuning.

Hardware aware optimization serves as the essential amplifier that converts theoretical compression gains into practical deployment realities. The gap between a model's theoretical FLOPs and its actual wall clock latency and energy consumption can exceed an order of magnitude without careful attention to memory access patterns, arithmetic unit utilization, and platform specific kernel implementations. The most effective deployment systems .. Shen et al.'s CPU INT4 runtime, T MAC for ARM edge processors, and LLMPi for Raspberry Pi .. all derive their practical effectiveness from co design of the compression scheme with the target platform's specific characteristics.

Looking ahead, the field's most pressing needs are: standardized energy measurement frameworks that enable objective cross study comparison; compression methods specifically designed for sub 7B models where current approaches are less effective; hardware software co design platforms that reduce the porting burden across the fragmented edge hardware landscape; and principled frameworks for designing coordinated multi technique optimization pipelines. The increasing availability of on device LLM inference .. across smartphones, embedded systems, and IoT platforms .. represents not merely a technical achievement but a fundamental shift in how artificial intelligence will be deployed in the real world: locally, privately, and without dependence on network connectivity. The research directions identified in this report will be essential to realizing the full potential of this transformation.

REFERENCES

[1] Anonymous Authors, "Sustainable LLM Inference for Edge AI: Evaluating Quantized LLMs for Energy Efficiency, Output Accuracy, and Inference Latency," ACM Transactions on Internet of Things, vol. 6, no. 4, Article 28, Nov. 2025.

[2] Y. Wang, T. Yang, X. Liang, G. Wang, H. Lu, X. Zhe, Y. Li, and L. Weitao, "Art and science of quantizing large scale models: A comprehensive overview," arXiv preprint arXiv:2409.11650, Sep. 2024.

[3] Y. Zheng, Y. Chen, et al., "A review on edge large language models: Design, execution, and applications," ACM Computing Surveys, vol. 57, no. 8, pp. 1–35, 2025.

[4] Q. Song, P. Liao, W. Zhao, Y. Wang, S. Hu, H. L. Zhen, N. Jiang, and M. Yuan, "Harnessing large language models locally: Empirical results and implications for AI PC," arXiv preprint, 2025.

[5] J. Xu, Z. Li, W. Chen, Q. Wang, X. Gao, Q. Cai, and Z. Ling, "On device language models: A comprehensive review," arXiv preprint arXiv:2409.00088, Aug. 2024.

[6] G. Yang, C. He, J. Guo, J. Wu, Y. Ding, A. Liu, H. Qin, P. Ji, and X. Liu, "LLMCBench: Benchmarking large language model compression for efficient deployment," Advances in Neural Information Processing Systems, vol. 37, pp. 87532–87544, 2024.

[7] R. Gong, Y. Ding, Z. Wang, C. Lv, X. Zheng, J. Du, Y. Yong, S. Gu, H. Qin, J. Guo, and D. Lin, "A survey of low bit large language models: Basics, systems, and algorithms," Neural Networks, p. 107856, 2025.

[8] Z. Mo, L. Wang, et al., "LUT tensor core: Lookup table enables efficient low bit LLM inference acceleration," arXiv preprint, Aug. 2024. (Also: Microsoft Research Blog, 'Advances to low bit quantization enable LLMs on edge devices,' Feb. 2025.)

[9] T. E. Jørgensen, "Resource efficient language models: Quantization for fast and accessible inference," arXiv preprint, May 2025.

[10] J. Hasan, "Optimizing large language models through quantization: A comparative analysis of PTQ and QAT techniques," arXiv preprint arXiv:2411.06084, Nov. 2024.

[11] B. Li, X. Wang, and H. Xu, "AWEQ: Post training quantization with activation weight equalization for large language models," arXiv preprint, Nov. 2023.

[12] H. Shen, H. Chang, et al., "Efficient LLM inference on CPUs," arXiv preprint, Nov. 2023.

[13] C. Lee, J. G. Jin, Y. Cho, and E. Park, "QEFT: Quantization for efficient fine tuning of LLMs," arXiv preprint arXiv:2410.08661, Oct. 2024.

[14] Y. Xu, L. Xie, et al., "QA LoRA: Quantization aware low rank adaptation of large language models," arXiv preprint, Sep. 2023.

[15] R. G. Dumitru, P. I. Cloțan, et al., "Change is the only constant: Dynamic LLM slicing based on layer redundancy," arXiv preprint, Nov. 2024.

[16] R. Agrawal, H. Kumar, and S. R. Lnu, "Efficient LLMs for edge devices: Pruning, quantization, and distillation techniques," in Proc. 2025 International Conference on Machine Learning and Autonomous Systems (ICMLAS), pp. 1413–1418, IEEE, 2025.

[17] J. Yin, J. Dong, et al., "ModuLoRA: Finetuning 2 bit LLMs on consumer GPUs by integrating with modular quantizers," arXiv preprint, Sep. 2023.

[18] T. Wallace and N. Ezzati Jivan, "Optimization strategies for enhancing resource efficiency in transformers and large language models," arXiv preprint, Jan. 2025.

[19] F. Liu, N. Yang, et al., "LCD: Advancing extreme low bit clustering for large language models via knowledge distillation," arXiv preprint, May 2025.

[20] C. Guo, F. Cheng, et al., "A survey: Collaborative hardware and software design in the era of large language models," arXiv preprint, Oct. 2024.

[21] Y. Li, J. Liu, H. Zhang, M. B. Narayanan, U. Sharma, S. Zhang, P. Hu, Y. Zeng, J. Raghuram, and S. Banerjee, "PalmBench: A comprehensive benchmark of compressed large language models on mobile platforms," arXiv preprint arXiv:2410.05315, Oct. 2024.

[22] M. Ardakani, J. Malekar, and R. Zand, "LLMPi: Optimizing LLMs for high throughput on Raspberry Pi," in Proc. Computer Vision and Pattern Recognition Conference, pp. 6379–6388, 2025.

[23] J. Lin, J. Tang, H. Tang, S. Yang, W. M. Chen, W. C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation aware weight quantization for on device LLM compression and acceleration," Proc. Machine Learning and Systems, vol. 6, pp. 87–100, MLSys 2024. (Best Paper Award)

[24] T. Dettmers, R. Svirschevski, V. Egiazarian, D. Kuznedelev, E. Frantar, S. Ashkboos, A. Borzunov, T. Hoefler, and D. Alistarh, "SpQR: A sparse quantized representation for near lossless LLM weight compression," in Proc. ICLR, 2024.

[25] Anonymous Authors, "A comprehensive evaluation of quantized instruction tuned large language models: An experimental analysis up to 405B," arXiv preprint arXiv:2409.11055, Sep. 2024.

[26] G. Shao, Y. Chen, et al., "OmniQuant: Omnidirectionally calibrated quantization for large language models," in Proc. ICLR, 2024.

[27] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post training quantization for large language models," in Proc. ICML, 2023.

[28] S. S. Girija, L. Arora, S. Kapoor, D. Pradhan, A. Raj, and A. Shetgaonkar, "Optimizing LLMs for resource constrained environments: A survey of model compression techniques," in Proc. IEEE 49th Annual COMPSAC, pp. 1657–1664, IEEE, 2025.

[29] A. Kaur, R. Sharma, et al., "Optimizing large language model deployment in edge environments," International Journal of Science and Research Archive, vol. 14, no. 3, pp. 1658–1669, 2025.

[30] W. Yao, R. Yazdani Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, "ZeroQuant: Efficient and affordable post training quantization for large scale transformers," Advances in Neural Information Processing Systems, vol. 35, 2022.

[31] J. Kim, M. Seo, and X. T. Nguyen, "Mixed INT4 INT8 LLM quantization via progressive layerwise assignment with dynamic sensitivity estimation," 2025 IEEE International Symposium on Circuits and Systems (ISCAS), 2025.

[32] O. Sagi et al., "Knowledge distillation and dataset distillation of large language models: Emerging trends, challenges, and future directions," Artificial Intelligence Review, Springer Nature, Nov. 2025.

[33] Anonymous Authors, "A comprehensive survey on knowledge distillation," Transactions on Machine Learning Research (TMLR), Sep. 2025.

[34] Anonymous Authors, "Survey on knowledge distillation for large language models: Methods, evaluation, and application," ACM Transactions on Intelligent Systems and Technology, 2024.

[35] Anonymous Authors, "Empowering edge intelligence: A comprehensive survey on on device AI models," ACM Computing Surveys, 2024.

[36] V. Chandra, "On Device LLMs: State of the Union, 2026," Technical Blog, 2026.

APPENDIX A: LIST OF ABBREVIATIONS

Abbreviation	Full Form
ACM	Association for Computing Machinery
ARM	Advanced RISC Machine
AWQ	Activation-Aware Weight Quantization
CNN	Convolutional Neural Network
DRAM	Dynamic Random-Access Memory
FFN	Feed-Forward Network
FLOPs	Floating-Point Operations
FP16	16-bit Floating-Point Precision
FP32	32-bit Floating-Point Precision
GPTQ	Generative Pre-trained Transformer Quantization
GPU	Graphics Processing Unit
IEEE	Institute of Electrical and Electronics Engineers
INT4	4-bit Integer Precision
INT8	8-bit Integer Precision
IoT	Internet of Things
KD	Knowledge Distillation
KV Cache	Key-Value Cache
LCD	Low-bit Clustering with Distillation
LLM	Large Language Model
LoRA	Low-Rank Adaptation
LR Score	Layer Redundancy Score
LUT	Lookup Table
MMLU	Massive Multitask Language Understanding
mpGEMM	Mixed-Precision General Matrix Multiplication
NPU	Neural Processing Unit
PTQ	Post-Training Quantization

QAT	Quantization-Aware Training
QA-LoRA	Quantization-Aware Low-Rank Adaptation
RISC-V	Reduced Instruction Set Computing - V
SoC	System on Chip
STE	Straight-Through Estimator
SpQR	Sparse Quantized Representation
TPU	Tensor Processing Unit
VRAM	Video Random-Access Memory
W4A16	4-bit Weights, 16-bit Activations
W8A8	8-bit Weights, 8-bit Activations

APPENDIX B: KEY MATHEMATICAL FORMULATIONS

C.1 Linear Quantization Mapping

The standard uniform linear quantization mapping converts a floating-point value x to an integer q via:

$$q = \mathrm{clamp}(\mathrm{round}(x / s) + z,\ q_min,\ q_max)$$

where:

- s = scale factor (step size between adjacent quantization levels)
- z = zero-point offset (integer, enabling asymmetric quantization)
- q_min, q_max = representable integer range (e.g., 0–15 for UINT4, −8 to 7 for INT4)
- clamp(·) clips values outside the representable range to the nearest bound

C.2 Dequantization

The approximate floating-point reconstruction from a quantized integer q is:

$$\hat{x} = s \cdot (q - z)$$

The quantization error $\varepsilon = x - \hat{x}$ is bounded by $|\varepsilon| \leq s/2$ for values within the representable range. For a b-bit symmetric quantization scheme, the step size is $\Delta = 2 \cdot |x_max| / (2^b - 1)$, and the mean squared quantization error is approximately $\Delta^2/12$ under a uniform distribution assumption.

C.3 Knowledge Distillation Loss

The combined distillation training objective is:

$$L_KD = (1 - \alpha) \cdot L_CE(y, z_S) + \alpha \cdot T^2 \cdot D_KL(\sigma(z_T/T) \parallel \sigma(z_S/T))$$

where:

- z_S, z_T = student and teacher logit vectors
- σ(·) = softmax function
- T = temperature (controls softness of probability distributions; typically 2–10 for LLMs)
- α = loss weighting coefficient (typically 0.5–0.9)
- L_CE = cross-entropy loss against ground-truth labels y
- D_KL = Kullback–Leibler divergence

C.4 Memory Footprint Estimation

Approximate memory required for weight storage at various precisions for a model with P parameters:

Precision	**Bytes per Parameter**	**Memory for 7B Model**	**Memory for 13B Model**
FP32 (32-bit)	4 bytes	~28 GB	~52 GB
FP16 (16-bit)	2 bytes	~14 GB	~26 GB
INT8 (8-bit)	1 byte	~7 GB	~13 GB
INT4 (4-bit)	0.5 bytes	~3.5 GB	~6.5 GB
INT2 (2-bit)	0.25 bytes	~1.75 GB	~3.25 GB

APPENDIX C: HARDWARE SPECIFICATIONS REFERENCE

The table below provides specifications for the key hardware platforms discussed in this report, providing context for deployment feasibility assessments and performance benchmarks cited throughout the text.

Table III. Hardware Specification Reference for Common Edge Platforms

Platform	Processor	RAM	Power Budget	Notes
Raspberry Pi 4 Model B	ARM Cortex-A72 (1.8 GHz, 4-core)	4 GB LPDDR4	5–7.5 W	Primary extreme-edge LLM benchmark (LLMPi, ACM IoT study)
Commodity x86 Laptop	Intel/AMD (e.g., i7-1355U)	8–16 GB DDR4/5	15–28 W (TDP)	Target for Shen et al. INT4 CPU runtime (18 tok/s LLaMA-13B)
NVIDIA RTX 3090/4090	GPU (10,496–16,384 CUDA cores)	24 GB GDDR6X	350–450 W	Target for ModuLoRA (65B fine-tuning in 24 GB VRAM)
Apple iPhone 15 Pro	Apple A17 Pro (6-core NPU)	8 GB LPDDR5	~3–5 W (inference)	Supports INT8 NPU; typical mid-range mobile edge target
Generic IoT Sensor Node	ARM Cortex-M4 (80–200 MHz)	256 KB – 4 MB Flash	< 100 mW	Most constrained class; sub-100 MB models only with INT2/INT4

Note: Power budgets represent typical inference power draw for the full system, not processor TDP alone. Actual values vary with workload, clock speeds, and power management settings. Memory capacities represent the configurations used in cited experiments; actual device variants may differ.

COMMENT BY EXTERNAL EXAMINER

Name of External Examiner:
Signature: